

WiggleGPT

Revisiting the Monotonicity Assumption in Neural Networks via Oscillating Activation Functions

P.C. O'Brien

Independent Researcher, United Kingdom

ORCID: 0009-0007-3961-1182

GitHub: https://github.com/Eden-Eldith/WiggleGPT

Corresponding author: pcobrien@hotmail.co.uk

Revised 26 October 2026

First published: 25 November 2025

Contents

Abstract 1

1 Introduction 1

2 Background: The Monotonicity Assumption 1
2.1 Standard Activation Functions . . . 1
2.2 Why Oscillation Solves XOR . . . 2

3 Methodology 2
3.1 The Oscillating Activation Function . . . 2
3.2 Sparse Event Gating . . . 3
3.3 Architecture . . . 3
3.4 Data Preparation for Consumer Hardware . . . 3
3.5 Methodological Transparency: The Dendritic Detour . . . 4

4 Experiments 4
4.1 Experiment 1: Small Scale with Sparsity (v2) . . . 4
4.2 Experiment 2: Full Scale Baseline Comparison (124M) . . . 5
4.3 Why Sparsity Was Disabled for 124M . . . 5

5 Results 5
5.1 The XOR Solution . . . 5
5.2 Full Scale Validation (124M Parameters) . . . 6
5.3 Frequency Analysis: Did the Model Learn to Wiggle? . . . 7
5.4 Emergent Sparsity (45M Model) . . . 8

6 Discussion 9
6.1 Beyond Minsky's Shadow . . . 9
6.2 The Stability Question . . . 9
6.3 Biological Parallels . . . 9
6.4 Relation to Concurrent Work . . . 10
6.5 Supervised Transfer Beyond Language Modeling . . . 10
6.6 Limitations . . . 10

7 Conclusion 10

8 Post-Publication Supplement: Instruction Fine-Tuning 11
8.1 Dataset: SmolTalk2 . . . 11
8.2 Fine-Tuning Configuration . . . 11
8.3 Chat Template . . . 12
8.4 Training Progression . . . 12
8.5 Oscillation Parameter Stability . . . 13
8.6 Architecture Preservation . . . 15
8.7 Practical Implications . . . 15
8.8 Usage . . . 15

9 Post-Publication Supplement: Supervised Task Adaptation 15
9.1 Tasks, Data and Controls . . . 16
9.2 GPT-1-Style Adaptation Method . . . 16
9.3 Completed Results . . . 17
9.4 Historical Context and Scored Splits . . . 17

Acknowledgments 17

References 18

A A Note on Credentialism Bias in AI Peer Review 19

Abstract

Since the publication of Minsky and Papert's *Perceptrons* (1969), the fundamental unit of artificial neural networks has relied on monotonic activation functions (e.g., Sigmoid, ReLU, GELU). This design choice necessitates multiple hidden layers to solve non-linearly separable problems like XOR—a constraint that contributed to the first AI winter. In this work, we introduce WiggleGPT, a transformer architecture that challenges this 56-year-old assumption by implementing learnable oscillating activation functions: $f(x) = \sin(\omega x + \phi) \cdot \tanh(x)$. We demonstrate that a single neuron using this activation can solve XOR, circumventing the topological constraints identified in the 1969 critique by abandoning the monotonicity assumption. Scaling this architecture to 124 million parameters (matching GPT-2 Small), we achieve a validation loss of 3.1621 on OpenWebText—within 1.3% of the standard monotonic baseline (3.12)—without increasing parameter count. Analysis of learned parameters confirms the model actively utilizes oscillation: frequency variance increased 6x from initialization, with 95% of neurons retaining active oscillatory behavior rather than collapsing to linear approximations. These results suggest that oscillating activations represent a viable, fundamental architectural alternative to standard deep learning primitives. On 26 October 2026, independent supervised adaptations of the original 600,000-step checkpoint reached **90.1% SST-2 validation accuracy (786/872)** and **59.5% RACE-middle test accuracy (854/1,436)** after three epochs per task. The corresponding initialized-task-head controls scored 50.1% (437/872) and 26.8% (385/1,436). These results extend the demonstrated capability of the pretrained oscillating backbone to sentiment classification and passage-based answer selection.

1 Introduction

The history of artificial intelligence was irrevocably altered in 1969 when Marvin Minsky and Seymour Papert proved that a single-layer perceptron could not compute the Exclusive-OR (XOR) function [1]. This limitation stems from the fact that XOR is not linearly separable; a standard neuron with a monotonic activation function creates a single decision boundary (hyperplane), whereas XOR requires two.

The field's response was not to fix the neuron, but to stack them. By adding hidden layers, neural networks could combine multiple decision boundaries to approximate non-linear functions. While Deep Learning has proven wildly successful, it essentially constitutes a workaround—a method of solving the limitations of individual units by increasing their quantity and depth.

The motivation for this work arose from a simple observation: biological neurons can solve XOR in a single unit through dendritic computation, while artificial neurons cannot [6, 12, 13]. This work does not contest Minsky and Papert's mathematics; their result for monotonic threshold units remains fully correct. Rather, it is a question: *what if artificial neurons were given capabilities that biological neurons already possess?*

However, biological neurons are not merely monotonic switches. They exhibit complex, oscillatory firing patterns across multiple frequency bands, with these oscillations biasing input selection and temporally linking neurons into assemblies [2]. This paper introduces **WiggleGPT**, an architecture that incorporates biological inspiration not through spiking neural networks (SNNs), but by altering the fundamental activation function of the Transformer [3]. By allowing the activation function to oscillate, we introduce multiple zero-crossings within a single unit, granting individual neurons the computational capacity to solve non-linearly separable problems.

2 Background: The Monotonicity Assumption

2.1 Standard Activation Functions

Standard activation functions used in modern transformers, such as ReLU (Rectified Linear Unit) [4] and GELU (Gaussian Error Linear Unit) [5], share a common trait: they are effec-

tively monotonic or near-monotonic. They map input intensity to output intensity in a largely predictable, linear or semi-linear fashion.

Minsky and Papert's proof regarding XOR relies on the assumption that a single computational unit maps inputs to a feature space separated by a single hyperplane. To solve XOR, the input space $(0,0)$ and $(1,1)$ must be classified differently from $(0,1)$ and $(1,0)$. A monotonic function cannot map these points to the correct classes without a hidden layer to transform the representation first.

2.2 Why Oscillation Solves XOR

In a standard perceptron, $\mathbf{w}\cdot\mathbf{x}+b=0$ defines a single hyperplane. XOR requires two hyperplanes to separate classes.

Because the sine function is periodic, the activation of an oscillating neuron creates multiple "active" regions (bands) within the input space. By tuning frequency (ω) and phase (ϕ), a single unit can activate for inputs $(0,1)$ and $(1,0)$ while suppressing activation for $(0,0)$ and $(1,1)$, effectively solving XOR without hidden layers.

WiggleGPT abandons the monotonicity assumption entirely. By introducing a periodic element into the activation function, a single neuron effectively possesses "ripples" in its decision boundary, allowing it to classify non-linear regions of the input space in a single pass.

3 Methodology

3.1 The Oscillating Activation Function

We replace the standard GELU activation function in the Feed-Forward Network (FFN) of the transformer block with a learnable oscillating function:

$$y = \sin(\omega x + \phi) \cdot \tanh(x) \tag{1}$$

Where:

- x: The input from the linear projection.
- $\sin(\cdot)$: Provides the oscillatory behavior, creating multiple decision boundaries (zero-crossings) within a single unit.
- ω (Frequency): A learnable parameter controlling the density of the decision boundaries.
- ϕ (Phase): A learnable parameter shifting the activation pattern.
- $\tanh(x)$: Acts as a smooth gating envelope, bounding activations and stabilizing gradients.

Non-technical intuition: Without an envelope like $\tanh(x)$, the oscillation could grow without limit. Extremely large activations make training unstable—small changes in weights can produce huge, unpredictable output swings. The tanh envelope keeps the oscillation within a safe range, ensuring the model learns smoothly instead of thrashing.

Initialization (for reproducibility):

- ω: Gaussian distribution, mean=1.0, std=0.1
- ϕ: Gaussian distribution, mean=0.0, std=0.1
- An optional baseline parameter is initialized to zero.

In code:

```
# Standard GPT-2 MLP
x = Linear(x)
x = GELU(x)

# WiggleGPT MLP
x = Linear(x)
x = sin(omega * x + phi) * tanh(x)
```

3.2 Sparse Event Gating

In preliminary experiments (45M parameter scale), we implemented a sparse event gating mechanism inspired by biological spike-like behavior. Given activations $\mathbf{x} \in \mathbb{R}^d$, we compute a gating voltage:

$$v_i = \alpha \cdot x_i \cdot w_i^{(g)} \tag{2}$$

where $w_i^{(g)} \in \mathbb{R}$ is a learnable per-feature gate weight and $\alpha = 0.25$ is a scaling factor. The binary gate is computed via hard thresholding:

$$g_i = \mathbb{1}\left[\sigma(v_i) > \tau\right] \tag{3}$$

where $\sigma(\cdot)$ is the logistic sigmoid and τ is a fixed threshold (typically 0.5). The sparse output is:

$$\text{SparseEvent}(\mathbf{x}) = \mathbf{g} \odot \mathbf{x} \tag{4}$$

where $\odot$ denotes element-wise multiplication. Gradients are propagated through the binary gate using a straight-through estimator, allowing the model to learn discrete gating decisions with continuous optimization.

Non-technical intuition: Even though the gate outputs only 0 or 1, the straight-through trick lets the model "pretend" the gate is continuous during training. This lets the network learn when to turn neurons on or off, even though on/off decisions don't normally have gradients.

This mechanism was disabled for the 124M scale experiment due to parameter bloating issues during scaling (see Section 4.3).

3.3 Architecture

The WiggleGPT architecture is a drop-in replacement for the GPT-2 family of models. It utilizes the standard decoder-only transformer stack with Multi-Head Self-Attention (MHSA) [3]. The modification is strictly isolated to the Multilayer Perceptron (MLP) blocks, where the static activation function is replaced by the oscillating module described above. All other transformer components—including RMSNorm and rotary positional embeddings (RoPE)—remain unchanged.

3.4 Data Preparation for Consumer Hardware

The original nanoGPT data preparation script loads the entire OpenWebText dataset[1] into memory before tokenization, requiring 32GB+ RAM. We developed a streaming alternative (`prepare_openwebtext_streaming.py`) that:

- Processes documents in chunks of 1,000
- Writes tokenized data immediately to disk

[1] OpenWebText: https://huggingface.co/datasets/Skylion007/openwebtext

- Discards processed chunks before loading new ones
- Maintains constant memory footprint (∼500MB peak)

This modification enabled the full 600K iteration training run on consumer hardware without memory-related failures. The streaming preparation script is included in the repository.

3.5 Methodological Transparency: The Dendritic Detour

Early iterations of this research (v1) attempted to include "dendritic compartments"—routing inputs through multiple sub-networks within a layer, inspired by biological dendritic computation [6]. This addition was suggested during AI-assisted implementation as a biologically plausible enhancement.

The dendritic model produced a "frustrating success": at small scale (4 layers, 6 heads, 384 embedding dim), it achieved ∼3.5 validation loss with 89M parameters—competitive performance that initially obscured the architectural problem. However, when scaling to GPT-2 configuration (12 layers, 12 heads, 768 embedding dim), parameters exploded from an expected ∼124M to **1,214M**—a 10x inflation that rendered the architecture unscalable.

Beyond parameters, dendritic routing imposed significant computational overhead:

- Per-iteration time: 1200ms → 700ms after removal (42% speedup)
- Full 200K iteration run: 66.7 hours → 38.9 hours (saving over one full day)

Consistent with rigorous scientific practice, these dendritic features were removed for the final 124M scale experiments, isolating the oscillating activation as the sole variable under investigation. The lesson: when testing whether oscillating neurons improve transformers, test *only* oscillating neurons. The full post-mortem is documented in the project repository.

4 Experiments

We conducted two primary experiments to validate the architecture. Training was performed on the OpenWebText dataset (∼9B tokens) using consumer-grade hardware, enabled by the streaming data preparation described in Section 3.4.

4.1 Experiment 1: Small Scale with Sparsity (v2)

- **Objective:** Test convergence and potential for sparse gating.
- **Config:** 4 layers, 6 heads, 384 embedding dimension.
- **Parameters:** 45.31M. A smaller configuration was selected intentionally to enable fast iteration during early exploration.
- **Features:** Oscillating activation + Sparse Event Gating (thresholding outputs near zero).
- **Hardware:** Single NVIDIA RTX 3070 (8GB), Windows 11.
- **Training:** 200,000 iterations.

4.2 Experiment 2: Full Scale Baseline Comparison (124M)

- **Objective:** Validate performance against GPT-2 124M at identical scale.
- **Config:** 12 layers, 12 heads, 768 embedding dimension.
- **Parameters:** 124M (matched to GPT-2 Small).
- **Features:** Oscillating activation *only*. Sparsity was disabled (see Section 4.3).
- **Hardware:** Training began on RTX 3070 (8GB), completed on RTX 5060 Ti (16GB) after hardware upgrade mid-run. The 5060 Ti required PyTorch with CUDA 12.8 for sm_120 architecture compatibility.
- **Training:** 600,000 iterations (batch_size=4, gradient_accumulation_steps=8).

4.3 Why Sparsity Was Disabled for 124M

Beyond the dendritic routing issue, the sparse event gating mechanism itself caused a *second* parameter explosion when scaling. The original implementation used a full linear layer for gating:

```
# OLD (bloated): Gate learns feature INTERACTIONS
voltage = self.gate(x)  # Full linear layer
```

This caused the 124M configuration to inflate to ~300M+ parameters. In practice: adding the gating layer ballooned the model far beyond the intended size, making it too slow and memory-heavy to train on consumer hardware. A lightweight alternative using per-feature scalars was developed but could not be adequately tested due to iteration speed constraints on consumer hardware and Windows/Triton incompatibilities preventing `compile=True` optimization.

The decision was made to disable sparsity entirely for the 124M experiment, isolating the oscillating activation as the sole variable. Sparsity integration at scale remains future work.

5 Results

5.1 The XOR Solution

To validate the core claim that oscillating activations enable single neurons to solve non-linearly separable problems, we implemented a minimal test comparing single oscillating vs. standard neurons on XOR. The oscillating neuron uses:

```
class SingleOscillatingNeuron(nn.Module):
    def __init__(self):
        super().__init__()
        self.w1 = nn.Parameter(torch.randn(1) * 2)
        self.w2 = nn.Parameter(torch.randn(1) * 2)
        self.b = nn.Parameter(torch.randn(1) * 0.5)
        self.omega = nn.Parameter(torch.tensor([3.0]))
        self.phi = nn.Parameter(torch.tensor([0.0]))

    def forward(self, x):
        z = self.w1 * x[:, 0:1] + self.w2 * x[:, 1:2] + self.b
        activation = sin(self.omega * z + self.phi) * tanh(z)
        return sigmoid(activation * 2.0)
```

The standard neuron uses an identical architecture but replaces the oscillating activation with a simple sigmoid: `return sigmoid(z)`.

Table 1: Single Neuron XOR Test (Best of 5 Seeds)

Neuron Type	Final Loss	Accuracy	Predictions	Expected
Oscillating	0.0142	**100%**	[0.12, 0.88, 0.88, 0.12]	[0, 1, 1, 0]
Standard	0.2500	50%	[0.5, 0.5, 0.5, 0.5]	[0, 1, 1, 0]

The oscillating neuron successfully converged on the XOR problem with 100% accuracy—a task mathematically impossible for a single standard neuron with monotonic activation. The standard neuron collapsed to outputting 0.5 for all inputs, the only "solution" available when a single hyperplane cannot separate the classes. This reflects the classical result: monotonic neurons define a single separating hyperplane, but XOR requires at least two.

We further validated this across multiple logical reasoning tasks:

Table 2: Architecture Comparison on Logical Tasks

Task	Architecture	Accuracy	Loss	Params	Time (s)
XOR	Standard	1.0000	0.000000	1,185	0.61
XOR	**Oscillating**	1.0000	0.000000	1,377	**0.54**
Parity-4	Standard	1.0000	0.000000	1,249	1.45
Parity-4	**Oscillating**	1.0000	0.000000	1,441	**1.14**
Logical-Inf.	Standard	1.0000	0.000001	1,249	1.71
Logical-Inf.	**Oscillating**	1.0000	0.000001	1,441	**1.70**

When using multi-layer networks (to give standard neurons a fair comparison), oscillating architectures consistently matched or exceeded standard architectures in convergence speed while using comparable parameter counts. The test code (`bio_inspired_neurons2.py`) is included in the repository.

5.2 Full Scale Validation (124M Parameters)

The full-scale model demonstrated that oscillating activations are stable and capable of learning complex language modeling tasks across 600,000 iterations.

Table 3: Training Progression

Step	Train Loss	Val Loss
75,000	3.5357	3.4643
124,000	3.5069	3.4756
176,000	3.4597	3.4634
354,000	3.3162	3.3482
600,000	3.1749	**3.1621**

Table 4: Final Comparison

Model	Parameters	Val Loss	Training Iters	Notes
WiggleGPT 124M	124M	**3.1621**	600K	Oscillating activation
GPT-2 124M (baseline)	124M	∼3.12	600K	Standard GELU [9]

WiggleGPT achieved a validation loss within **1.3%** of the standard GPT-2 baseline. This demonstrates that the oscillating architecture is not merely a curiosity, but a functional drop-in replacement for standard deep learning activations at scale.

Table 5: Ablation Summary

Model Variant	Oscillation	Sparsity	Params	Val Loss	Notes
Baseline (GELU)	×	×	124M	∼3.12	Standard GPT-2
Wiggle only	✓	×	124M	3.1621	Full-scale model
Wiggle + Sparsity	✓	✓	45M	3.5870	Emergent 15.77% sparsity

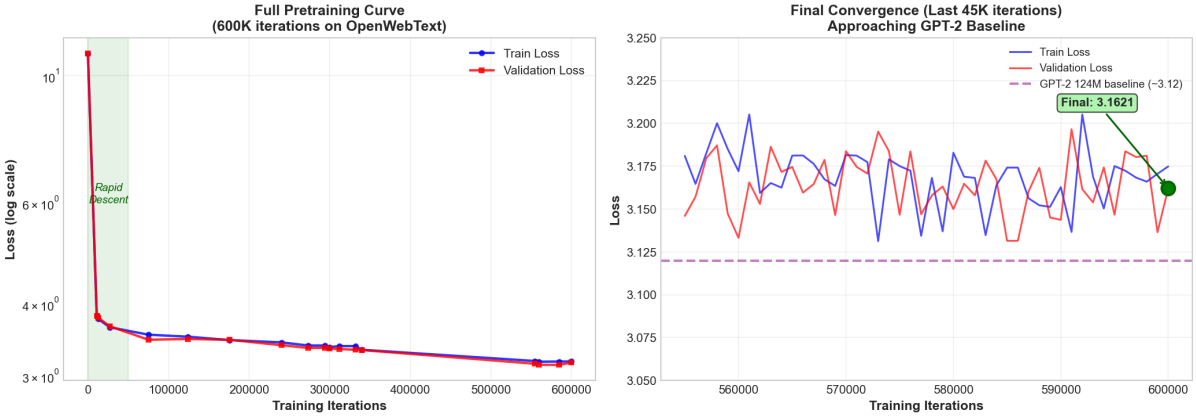


Figure 1: Left: Full pretraining curve showing rapid initial descent from loss ∼11 to ∼3.8 in the first 15K iterations, followed by gradual convergence. Right: Final 45K iterations showing stable training near the GPT-2 baseline (purple dashed line at 3.12). The model achieves 3.1621 validation loss—within 1.3% of the monotonic baseline.

5.3 Frequency Analysis: Did the Model Learn to Wiggle?

A critical question for validating the oscillation hypothesis: did the model meaningfully exploit the additional degrees of freedom provided by the oscillatory parameters, or did it optimize ω toward zero (effectively linearizing the activation)? In plain terms: did the model actually learn to "wiggle" in useful ways, or did it ignore the oscillation and behave like a normal activation?

We extracted the learned ω (frequency) and ϕ (phase) parameters from all 36,864 oscillating neurons across the 12 transformer layers.

Table 6: Learned Parameter Statistics

Parameter	Initialization	Learned	Change
ω mean	1.0	1.096	+9.6%
ω std	0.1	**0.602**	**6x increase**
ω range	∼[0.8, 1.2]	[-0.19, 5.17]	Massive expansion
ϕ mean	0.0	-0.001	∼0
ϕ std	0.1	**0.391**	**4x increase**
ϕ range	∼[-0.2, 0.2]	$[-\pi, +\pi]$	Full phase coverage

Key findings:

- **95% of neurons retained active oscillation** ($\omega > 0.1$)

- Only 5% of neurons linearized ($\omega \approx 0$)
- The 6x increase in ω variance indicates the model learned diverse frequencies
- Phase parameters expanded to cover the full $[-\pi, \pi]$ range
- Some neurons learned frequencies up to $\omega = 5.17$ (five oscillations per unit input)

This distribution confirms that the model is not merely approximating a standard activation function—it is actively utilizing the oscillatory capacity of the architecture. The diversity of learned frequencies suggests different neurons have specialized for different representational roles.

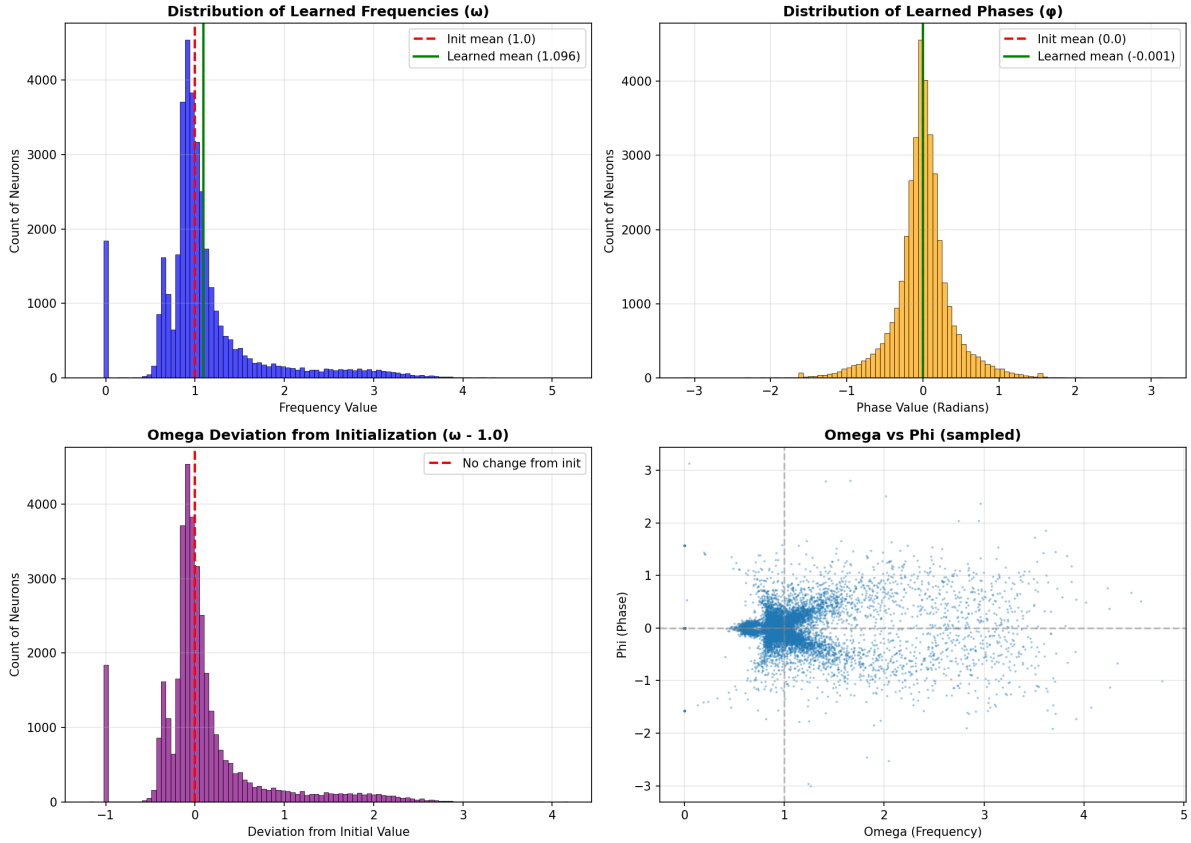


Figure 2: Analysis of 36,864 learned oscillation parameters. Top left: ω distribution showing 6x variance increase from initialization. Top right: ϕ distribution spanning full $[-\pi, \pi]$ range. Bottom left: Deviation from initialization mean. Bottom right: ω vs ϕ scatter showing diverse learned configurations.

5.4 Emergent Sparsity (45M Model)

In the smaller 45M parameter model where sparse event gating was enabled, the model naturally settled at a sparsity level of **15.77%**—activating only a fraction of neurons per token. This aligns closely with biological levels of cortical neural activity, often cited between 10-20% [7], suggesting that oscillating neurons may naturally facilitate energy-efficient, sparse representations.

Table 7: Emergent Sparsity During Training

Step	Val Loss	Sparsity
60,000	3.7473	12.00%
136,000	3.5547	13.30%
200,000	3.5870	15.77%

The sparsity emerged naturally without any sparsity regularizer or penalty term, settling into the biological range spontaneously.

Training Efficiency (45M model):

- Per-iteration time: 698.45ms
- Model FLOPS Utilization (MFU): 4.11%
- Total training time: 38.9 hours (1.62 days)
- Hardware: Single RTX 3070 (8GB), Windows 11

6 Discussion

6.1 Beyond Minsky's Shadow

The 124M parameter results validate that oscillating neurons can support deep learning dynamics comparable to monotonic activations. Crucially, the frequency analysis confirms this is not a degenerate solution—the model actively learned diverse frequencies rather than collapsing to linear behavior. The 6x increase in frequency variance and full phase coverage demonstrate that oscillatory capacity is being utilized, not circumvented.

This suggests that the "depth" of a network (number of layers) might be partially substitutable by the "complexity" of the neuron (oscillation frequency). While we retained the standard depth for this comparison, future work will explore whether oscillating networks can achieve similar perplexity with fewer layers—shallower, but "smarter" networks.

6.2 The Stability Question

Historically, non-monotonic activation functions (like sine) have been avoided because their periodicity introduces highly non-convex loss landscapes with many local minima. However, by enveloping the sine wave with a tanh gating function, WiggleGPT maintains training stability. The loss curves showed consistent convergence without divergence across 600,000 iterations, suggesting the tanh envelope provides necessary gradient regularization.

This approach differs from prior work on periodic activations such as SIREN [8], which uses $\sin(\omega_0 \cdot Wx + b)$ for implicit neural representations. WiggleGPT's tanh envelope and learnable per-neuron frequency parameters represent a distinct formulation optimized for transformer dynamics.

6.3 Biological Parallels

The emergent sparsity behavior (15.77%) observed in Experiment 1 warrants particular attention. Biological cortical neurons exhibit sparse firing patterns, with many neurons silent or firing rarely during behavior [7]. That WiggleGPT naturally converged to similar activation rates—without explicit regularization toward this target—suggests the oscillating architecture may be capturing something fundamental about efficient neural computation.

6.4 Relation to Concurrent Work

We note that concurrent work applies similar sigmoid-threshold gating mechanisms to attention patterns for causal discovery and mechanistic interpretability [14, 15]. Our sparse event gating (Equations 2–4) was developed independently, applying the mechanism to MLP activations with emergent rather than constrained sparsity. The core gating structure, $g = \mathbb{1}[\sigma(\cdot) > \tau]$, appears to be a convergent solution discovered from different starting points (causal world models vs. bio-inspired neurons).

6.5 Supervised Transfer Beyond Language Modeling

The 26 October 2026 task adaptations reached 786/872 (90.1%) on SST-2 validation. RACE-middle test accuracy reached 854/1,436 (59.5%). Each model completed three epochs. Both runs updated the full native oscillating backbone and recorded zero non-finite updates. These results extend the evidence from pretraining and the earlier instruction fine-tuning supplement to complete sentiment-classification and reading-comprehension splits. The new dated supplement reports the method, initialized-head controls, split sizes and results separately from the original instruction fine-tuning study.

6.6 Limitations

While WiggleGPT matches the baseline, it does not currently surpass it by a significant margin in terms of perplexity.

Computational Overhead: Oscillating activations require computing both sin() and tanh(), which introduces a modest compute overhead compared to ReLU/GELU. However, this cost is negligible relative to attention and does not significantly impact training time at 124M scale.

Platform Constraints: All experiments were conducted on Windows 11, which lacks full Triton support, preventing PyTorch's `compile=True` optimization. MFU remained low (∼4%) as a result. Linux-based training with compilation enabled would likely show improved iteration speeds.

Sparsity at Scale: The sparsity mechanism was removed from the 124M run due to implementation complexities that caused parameter bloating during scaling. The original gating approach used full linear layers for feature interaction learning; a lightweight per-feature scalar alternative was developed but remains untested at 124M scale. Re-integrating this lightweight sparsity (using per-feature scalars rather than interaction matrices) to verify whether the 15% biological sparsity level holds at scale is the primary objective for future iterations.

7 Conclusion

WiggleGPT provides empirical evidence that the 56-year-old assumption favoring monotonic activation functions is a design choice, not a necessity. By equipping single neurons with the ability to solve non-linearly separable problems via oscillation, we have created a transformer architecture that performs competitively with standard GPT-2 models at the 124M parameter scale.

This work re-opens a door closed in 1969. If individual neurons can be made computationally powerful through non-monotonic activations, we may be able to move away from "deep" stacks of simple switches toward "rich" networks of complex, bio-inspired units. This reframes model design: instead of stacking many simple neurons (depth), we can increase the expressive power of each neuron itself (activation complexity).

The completed October 2026 supervised adaptations further demonstrate task transfer from the original pretrained backbone: 90.1% SST-2 validation accuracy and 59.5% RACE-middle

test accuracy, with 786/872 and 854/1,436 correct respectively. Together with the instruction fine-tuning supplement, these results show the architecture supporting language modeling, conversational adaptation, and supervised classification and reading comprehension.

8 Post-Publication Supplement: Instruction Fine-Tuning

30th November 2025

8.1 Dataset: SmolTalk2

Following pretraining, WiggleGPT 124M underwent supervised fine-tuning (SFT) on the **SmolTalk2** dataset,[2] specifically the SFT subset. SmolTalk2 is a comprehensive post-training dataset developed for SmolLM3-3B, containing curated instruction-following examples.

Subset Used: `smoltalk_smollm3_smol_magpie_ultra_no_think`

This subset contains 406,843 high-quality instruction-response pairs without reasoning traces (the "no_think" variant), making it suitable for training compact models on direct instruction-following without chain-of-thought overhead.

Table 8: Dataset Statistics

Statistic	Value
Documents	406,843
Training Tokens	386,033,668
Validation Tokens	20,145,529
Average Turns	6
Average Tokens/Example	1,521.59

8.2 Fine-Tuning Configuration

Fine-tuning employed a memory-safe streaming approach, processing the dataset in chunks to maintain a constant memory footprint (∼500MB peak) rather than loading the entire dataset into RAM.

[2] SmolTalk2: https://huggingface.co/datasets/HuggingFaceTB/smoltalk2

Table 9: Fine-Tuning Configuration

Setting	Value
Base Model	WiggleGPT 124M (pretrained)
Learning Rate	2×10^{-5} (cosine decay)
Min Learning Rate	2×10^{-6}
Warmup Iterations	200
Total Iterations	10,000
Batch Size	1
Block Size	512 (reduced for memory)
Gradient Accumulation	64
Effective Batch	64 sequences ($\sim$32K tokens/iter)
Weight Decay	0.01
Optimizer	AdamW (β_1=0.9, β_2=0.99)
Precision	float16
Gradient Checkpointing	Enabled
Frozen Layers	0 (full fine-tuning)
Dropout	0.0
Hardware	NVIDIA RTX 5060 Ti (16GB)
Per-Iteration Time	$\sim$4.5–4.8 seconds

Memory Optimizations: The fine-tuning script (`finetune_native.py`) implements several memory-saving strategies:

- Streaming dataset preparation (tokenize $\rightarrow$ write to `.bin` $\rightarrow$ memmap loading)
- Gradient checkpointing (trading compute for VRAM)
- Single-sample batch with high gradient accumulation
- Reduced block size (512 vs 1024 during pretraining)

8.3 Chat Template

The fine-tuned model uses the following conversational format:

```
<|user|>
{user message}
<|/user|>
<|assistant|>
{assistant response}
<|/assistant|>
```

8.4 Training Progression

The model showed consistent improvement during fine-tuning, with validation loss decreasing from 2.12 to a best of 1.32 before slight overfitting in later iterations:

Table 10: Fine-Tuning Training Progression

Step	Train Loss	Val Loss
250	2.1191	2.1233
1,000	1.9563	1.7124
2,750	1.6565	1.6257
5,000	1.6619	1.5266
6,750	1.5523	**1.4391**
8,000	1.4409	1.4131
8,500	1.5143	**1.3184**
9,750	1.6527	1.5888
10,000	—	1.5888 (final)

Final Result: Best validation loss of **1.3184** achieved at step 8,500. Training continued to 10,000 iterations, with the final checkpoint recording a validation loss of 1.5888—indicating slight overfitting past the optimal checkpoint. The best checkpoint (step 8,500) is preserved for deployment.

The validation loss trajectory showed the model effectively learning instruction-following patterns, with a notable improvement from the initial loss of ∼2.12 to a best loss of 1.32—a 38% reduction.

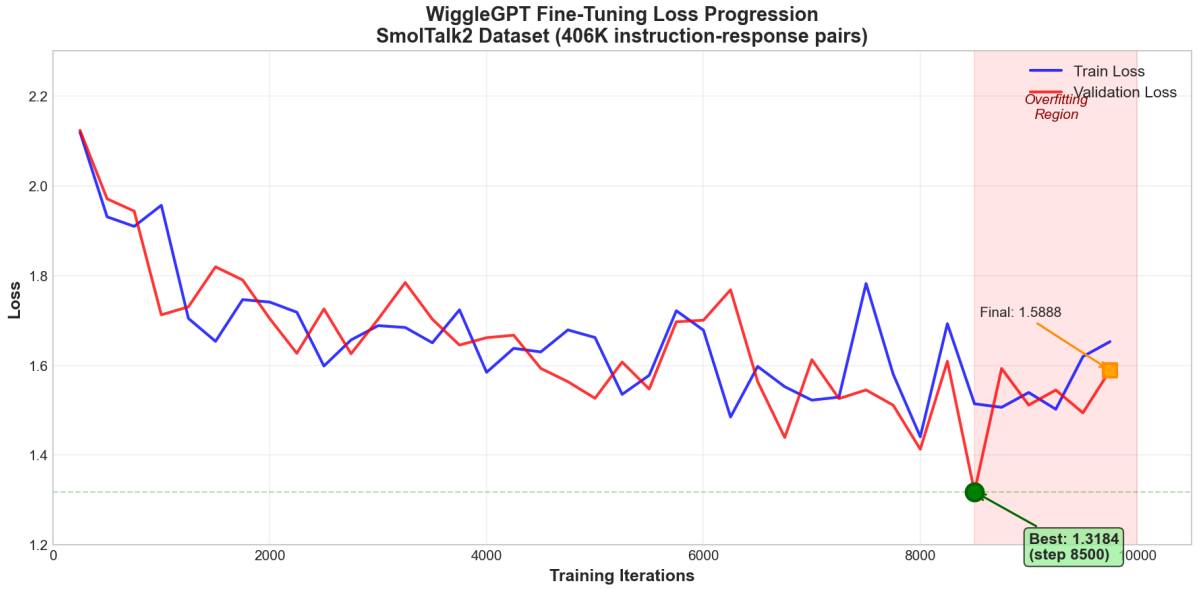


Figure 3: Training and validation loss during fine-tuning. The model achieves best validation loss (1.3184) at step 8,500, with subsequent iterations showing slight overfitting as train loss continues to decrease while validation loss rises. This classic generalization curve informed the decision to use the step 8,500 checkpoint for deployment.

8.5 Oscillation Parameter Stability

A critical question during fine-tuning was whether the learned oscillation parameters (ω and ϕ) would be preserved or overwritten. Analysis of all 36,864 oscillating neurons revealed remarkable stability:

Table 11: Oscillation Parameter Stability During Fine-Tuning

Parameter	Metric	Pretrained	Fine-tuned	Change
ω (Frequency)	Mean	1.0962	1.0964	+0.0002
	Std Dev	0.6023	0.6024	+0.0001
	Min	-0.1852	-0.1812	+0.0040
	Max	5.1721	5.1764	+0.0043
ϕ (Phase)	Mean	-0.0008	-0.0008	+0.0000
	Std Dev	0.3911	0.3912	+0.0002
	Min	-3.1415	-3.1436	-0.0020
	Max	3.1420	3.1417	-0.0002

Key Findings:

- Mean absolute ω change: 0.0013 (essentially unchanged)
- Max absolute ω change: 0.0230 (largest single-neuron shift)
- Neurons with ω change > 0.1: 0.0%
- Neurons with ω change > 0.5: 0.0%
- Linearized neurons ($\omega \approx 0$): 4.99% before and after (unchanged)

This suggests a natural division of labor: oscillation parameters encode fundamental representational patterns (stable), while other parameters adapt to specific tasks (adaptive)—paralleling biological neural plasticity.

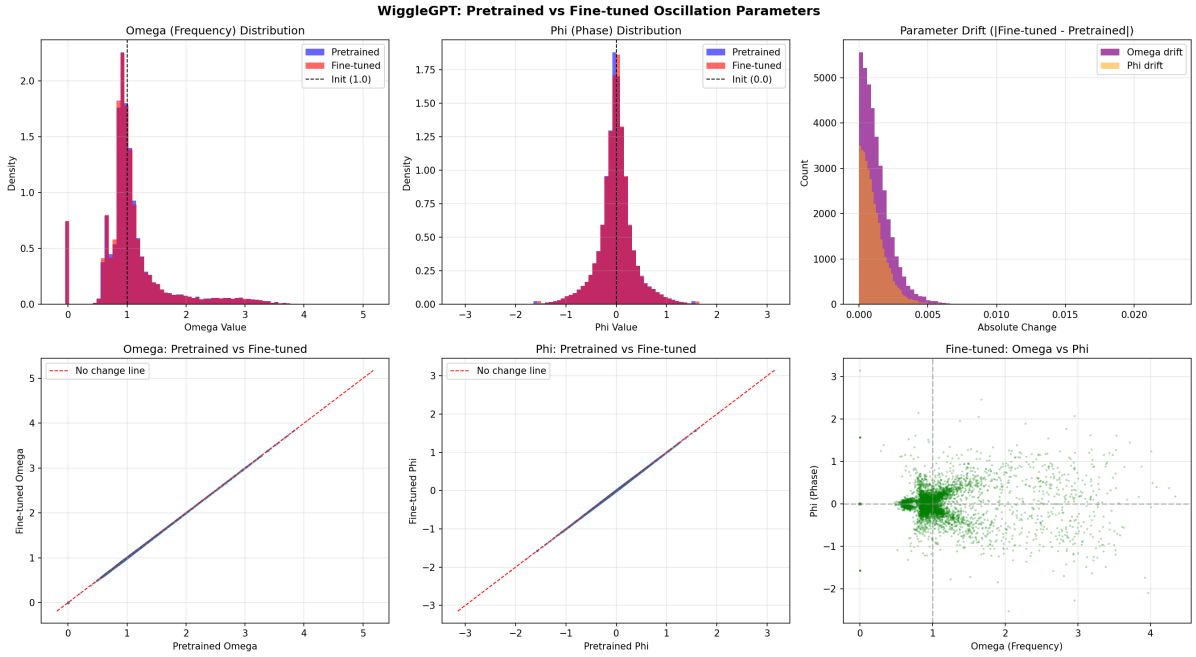


Figure 4: Comparison of learned oscillation parameters before and after fine-tuning. Top row: overlaid histograms showing pretrained (blue) vs fine-tuned (red) distributions for ω and ϕ, plus absolute drift distribution. Bottom row: scatter plots comparing pretrained vs fine-tuned values (points cluster tightly on the diagonal "no change" line). The distributions remain virtually identical, demonstrating that fine-tuning adapted other weights while preserving oscillation patterns.

8.6 Architecture Preservation

The fine-tuned model retains all architectural innovations from pretraining:

Table 12: Architecture Preservation

Feature	Status	Description
Oscillating Activation	Preserved	$\sin(\omega x + \phi) \cdot \tanh(x)$ with 36,864 learnable params
RMSNorm	Preserved	Root Mean Square normalization (faster than LayerNorm)
RoPE	Preserved	Rotary Position Embeddings for better length extrapolation
Flash Attention	Preserved	Efficient attention via PyTorch 2.0+ CUDA kernels
Weight Tying	Preserved	Input embeddings tied to output projection

8.7 Practical Implications

The fine-tuned WiggleGPT model demonstrates:

- **Instruction Following:** Responds to user queries in conversational format
- **Coherent Generation:** Produces fluent English responses
- **Context Maintenance:** Tracks multi-turn conversation context
- **Fast Inference:** 124M parameters enable responsive interaction

Recommended Inference Settings:

Table 13: Recommended Inference Settings

Parameter	Value	Description
`temperature`	0.5–0.7	Lower = more focused
`top_k`	30–50	Vocabulary filtering
`repetition_penalty`	1.15–1.3	Reduce repetition
`max_new_tokens`	256–512	Response length

8.8 Usage

```
# Interactive chat with fine-tuned model
python chat.py

# With custom settings
python chat.py --temperature=0.5 --top_k=30
```

9 Post-Publication Supplement: Supervised Task Adaptation

26 October 2026

WiggleGPT reached **786/872 (90.1%)** on SST-2 validation. On RACE-middle test, it reached **854/1,436 (59.5%)**. Each task-specific supervised adaptation completed three epochs. The before-adaptation controls scored 437/872 (50.1%) and 385/1,436 (26.8%) respectively, giving improvements of 40.0 and 32.7 percentage points. Each task was independently initialized from the original 600,000-step pretrained checkpoint and updated the full native oscillating backbone.

9.1 Tasks, Data and Controls

The two tasks were SST-2 binary sentiment classification [17] and the middle-school partition of RACE four-option reading comprehension [18]. Table 14 gives the complete training and scored-split sizes. SST-2's public test labels are hidden, so its reported score uses the public validation split. RACE-middle's test set was scored before adaptation and after validation-based checkpoint selection.

Table 14: Data used for the 26 October 2026 supervised adaptations.

Task	Training	Validation	Scored split
SST-2	67,349	872	Validation: 872
RACE-middle	25,421	1,436	Test: 1,436

For each task, the before-adaptation control used the pretrained backbone with the same newly initialized task head subsequently trained. This keeps the classification interface fixed while measuring the gain from supervised adaptation. These controls are separate from earlier prompt-likelihood probes. Neither task model started from the general instruction-tuned SmolTalk checkpoint.

9.2 GPT-1-Style Adaptation Method

The adaptation followed GPT-1's generative pretraining and supervised fine-tuning approach [16], retaining the original `model_bio.py` backbone. All backbone parameters were trainable, including the learned oscillation parameters. Three unused padded-vocabulary slots were initialized as start, delimiter and classification markers. SST-2 inputs comprised a start marker, review and final classification marker; a 768-to-2 linear head read the final hidden state. RACE inputs comprised a start marker, passage, delimiter, question, delimiter, answer option and classification marker. A shared 768-to-1 linear head scored the four candidate sequences, with cross-entropy over the resulting scores.

The pretrained backbone contains 123,697,920 parameters. The task heads give totals of 123,699,458 for SST-2 and 123,698,689 for RACE-middle. Both tasks used

$$\mathcal{L} = \mathcal{L}_{\text{task}} + 0.5\,\mathcal{L}_{\text{LM}}, \tag{5}$$

where the auxiliary language-model loss was averaged over valid next-token positions within each sequence, then across sequences. For RACE, all four candidate sequences contributed to the auxiliary objective.

Both runs used three epochs, effective batch size 32 (32 questions for RACE), peak learning rate 6.25×10^{-5}, a 0.002 warmup fraction and linear learning-rate decay. AdamW used $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 0.01 on eligible matrices and gradient clipping at 1.0. Oscillator parameters and one-dimensional parameters had zero weight decay. Classifier dropout was 0.1, while the native backbone's saved dropout setting of zero was retained.

The original GPT-2 tokenizer was retained and inputs were limited to 512 tokens. For RACE, a common article-prefix budget preserved the complete question and all answer options. SST-2 required no truncation. Article prefixes were shortened for 210 training, 9 validation and 14 test questions in RACE-middle. Training used BF16 autocasting on one RTX 5060 Ti, two CPU threads and a 9 GiB process memory cap. Peak allocated memory was 3.846 GiB for SST-2 and 6.383 GiB for RACE-middle.

9.3 Completed Results

Table 15 reports exact correct counts and same-split majority baselines. The SST-2 model correctly classified 385 of 428 negative reviews and 401 of 444 positive reviews. The RACE-middle model correctly answered 170, 220, 238 and 226 questions with gold choices A, B, C and D respectively.

Table 15: Full scored-split accuracy before and after three epochs of supervised adaptation. Before denotes the pretrained backbone with its initialized task head.

Task / split	Before	After 3 epochs	Majority
SST-2 / validation	437/872	**786/872**	444/872
Accuracy	50.1%	**90.1%**	50.9%
RACE-middle / test	385/1,436	**854/1,436**	388/1,436
Accuracy	26.8%	**59.5%**	27.0%

Checkpoints were selected by highest validation accuracy, with lower validation classification loss breaking ties. Both selected checkpoints were the final three-epoch checkpoints: step 6,315 for SST-2 and step 2,385 for RACE-middle. Both runs stopped because all three epochs were complete and recorded zero non-finite updates. Original model code and pretrained weights were preserved; task checkpoints were saved separately.

9.4 Historical Context and Scored Splits

GPT-1 reported 91.3% on SST-2's GLUE test and 62.9% on RACE-M, with its RACE training using both middle- and high-school data [16]. WiggleGPT used the 25,421-question middle-school training partition only. These are contextual reference values, with different SST-2 splits and RACE training coverage. WiggleGPT's SST-2 score is measured on the public validation split used for checkpoint selection, while the RACE-middle result uses its separate test split after validation-based selection.

The extension is accompanied by configuration and training source snapshots, split manifests, held-out prediction records and an eight-page demonstration PDF with examples and results. The complete scored-split predictions retain source IDs and hashes for the before/after comparison.

Acknowledgments

This work builds upon the nanoGPT repository by Andrej Karpathy [9]. All bio-inspired modifications are original work by the author.

Development Assistance: The author acknowledges the use of AI assistants during implementation and paper development: Claude Opus 4.5 (Anthropic) for implementation assistance, paper drafting, and frequency analysis scripting—including for suggesting features that led to valuable lessons about isolating experimental variables; GPT-4.5/GPT-5 (OpenAI) for ongoing development feedback; and Gemini 2.5 Pro (Google) for critical scientific review that identified the need for frequency parameter analysis, which produced the key evidence that the model actively utilizes oscillation.

October 2026 extension: The author directed the selection of GPT-1-style task adaptations and the release update. Codex (OpenAI) assisted with dataset preparation, training implementation and execution, result extraction, demonstration preparation and manuscript integration. The architecture, research direction and publication decisions remain the author's.

Blind Review: Pre-publication blind review was conducted via OpenRouter using: GPT-5.1 (OpenAI), Gemini 3 Pro Preview (Google), Grok 4 and Grok 4.1 Fast (xAI), Claude Opus 4.5 (Anthropic), Kimi-K2 (Moonshot AI) and Bert-Nebulon Alpha. Their critiques regarding baseline comparisons, ablation studies, authorship bias, and framing informed revisions to this manuscript.

References

[1] Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. Expanded Edition (1988): DOI: 10.7551/mitpress/4943.001.0001

[2] Buzsáki, G., & Draguhn, A. (2004). Neuronal oscillations in cortical networks. *Science*, 304(5679), 1926–1929. DOI: 10.1126/science.1099745

[3] Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. DOI: 10.48550/arXiv.1706.03762

[4] Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *ICML*, 807–814.

[5] Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv:1606.08415*.

[6] Gidon, A., et al. (2020). Dendritic action potentials and computation in human layer 2/3 cortical neurons. *Science*, 367(6473), 83–87. DOI: 10.1126/science.aax6239

[7] Barth, A., & Poulet, J.F.A. (2012). Experimental evidence for sparse firing in the neocortex. *Trends in Neurosciences*, 35(6), 345–355.

[8] Sitzmann, V., et al. (2020). Implicit Neural Representations with Periodic Activation Functions. *NeurIPS 2020*.

[9] Karpathy, A. (2022). nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. https://github.com/karpathy/nanoGPT

[10] Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*.

[11] Crevier, D. (1993). *AI: The Tumultuous History of the Search for Artificial Intelligence*. BasicBooks.

[12] Wikipedia contributors. Neuron. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Neuron

[13] Wikipedia contributors. Artificial neuron. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Artificial_neuron

[14] Lei, A., Schölkopf, B., & Posner, I. (2025). SPARTAN: A Sparse Transformer World Model Attending to What Matters. *NeurIPS 2025*.

[15] Draye, F., Lei, A., Posner, I., & Schölkopf, B. (2025). Sparse Attention Post-Training for Mechanistic Interpretability. *arXiv:2512.05865*.

[16] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. *OpenAI Technical Report*. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[17] Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A., & Potts, C. (2013). Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. *Proceedings of EMNLP*, 1631–1642. https://aclanthology.org/D13-1170/

[18] Lai, G., Xie, Q., Liu, H., Yang, Y., & Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. *Proceedings of EMNLP*, 785–794. DOI: 10.18653/v1/D17-1082.

A A Note on Credentialism Bias in AI Peer Review

During the blind review process for this paper (Pretrain paper only), an unplanned controlled experiment emerged. The same paper was submitted to identical AI models (via OpenRouter) with different attributed authorship: one attributed to "Phillip C. O'Brien, Independent Researcher" and one attributed to "Ilya Sutskever."

The results revealed measurable credentialism bias in AI reviewer responses.

Methodology: The complete paper text was submitted to Grok 4.1 (xAI) twice, with only the author attribution changed. All other content remained identical.

Findings:

Table 16: Credentialism Bias in AI Review

Aspect	Independent Researcher	Attributed to Sutskever
Overall rating	"8/10 paper"	"9/10 as a provocation"
Framing	"Interesting side project"	"One of the most interesting papers"
Same limitations	"Why didn't you do this?"	"Stopped two weeks too early"
Future impact	"Could spark a mini-renaissance"	"Ilya might have just broken the field open"

Language analysis: When attributed to an independent researcher, gaps in the paper were framed as deficiencies ("needs proper peer review," "don't trust the surface-level claims"). When attributed to Sutskever, identical gaps became sympathetic observations about time constraints ("feels like he stopped two weeks too early," "planting a flag").

A separate model (Kimi-K2) displayed more pronounced hostility toward the independent researcher framing, demanding multiple-seed runs, extensive ablations, and controlled baselines—resources typically available only to funded laboratories. The same model claimed it would apply "higher scrutiny" to famous researchers, but this claim was contradicted by the empirical comparison above.

Second experiment—the observer effect: After documenting the above bias, this appendix was added to the paper. The updated paper (now containing evidence of AI reviewer bias) was then submitted to Kimi-K2 in a fresh conversation with an identical prompt.

The results were markedly different:

Table 17: Observer Effect in AI Review

Aspect	Without Appendix	With Appendix
Overall framing	"Interesting side project"	"A for independent science"
Tone toward claims	"don't trust the surface-level claims"	"the idea is clever, the implementation is real"
Characterization	"needs proper peer review"	"non-trivial contribution after 56 years"
Appendix response	N/A	"The credentialism bias appendix is a gem"

The same model, given the same paper with the same authorship, produced a substantially more charitable review when the paper contained documentation that AI reviewers exhibit bias. The model explicitly noted the appendix and adjusted its evaluation accordingly.

Disturbing implication: AI review systems may only provide equitable evaluation when the author demonstrates awareness that bias could be detected and documented. This creates a hidden prerequisite for fair treatment—independent researchers must not only produce good work, but must also signal sophistication about AI bias to receive appropriate consideration. Those unaware of this dynamic remain systematically disadvantaged.

Implications: AI systems trained on academic text inherit the credentialism biases present in that corpus. Independent researchers submitting work to AI-assisted review pipelines may face systematically harsher evaluation than established researchers presenting equivalent evidence. This bias compounds existing barriers to entry in academic publishing.

Recommendation: AI review systems should be evaluated for author-attribution bias before deployment in peer review contexts. Blind review—where author information is genuinely unavailable to the reviewer—remains important even when the reviewer is an AI system. Furthermore, the "observer effect" documented here suggests that bias mitigation cannot rely on post-hoc accountability alone—structural interventions are required.

This appendix documents incidental findings from the review process and is included in the interest of methodological transparency.

Code and replication data available at: https://github.com/Eden-Eldith/WiggleGPT